Age Recovery: contexto completo del proyecto

Resumen de lo construido, decisiones tomadas y problema en curso. Estado al 7 de noviembre de 2026, 3:40 AM. Alumno: Vladimir Emanuel Alcalá Félix. Demo y prueba: martes próximo.

1. Qué es el proyecto

Ecosistema de seguimiento posprocedimiento para Age Recovery, una clínica de medicina estética en San Luis Río Colorado, Sonora. Materia: Desarrollo de Dispositivos Inteligentes, Parcial 2. Piezas:

- App móvil (Expo, iPhone XS con Expo Go).
- Reloj inteligente (Wear OS, emulador de Android Studio).
- Backend propio (Node + Express + MySQL).
- Chatbot de la clínica (Azure, Microsoft Foundry), ya existente.

La app y el reloj hablan con el mismo backend. El chatbot es público y no recibe token ni datos del paciente.

Ruta base en la PC: `C:\Users\Vlad-pc\Desktop\UTSLRC\INGIENERIA\CUATRIMESTRE 9-3\DESARROLLO DISPOSITIVOS INTELIGENTES\PARCIAL 2\EXAMEN`

Contenido actual de esa carpeta:

- `3-AppMovil-Expo`
- `4-Backend`
- `agerecovery-chatbot-foundry-main` (versión local con respuestas cortas, sin publicar)
- `recuperacion_app.sql`
- `sos-wear-os-main` (proyecto del reloj, ubicado en la misma carpeta EXAMEN)

2. Arquitectura

Pieza	Tecnología	Dónde corre	Cómo se alcanza
App móvil	Expo SDK 58, React Native, TypeScript	iPhone XS con Expo Go	QR de `npx expo start`
Backend	Node, Express, mysql2, JWT 8 h, helmet, rate limit	PC, puerto 3000, escucha en 0.0.0.0	`http://192.168.1.14:3000` desde el celular; `http://10.0.2.2:3000` desde el emulador
Base de datos	MySQL 8, esquema `recuperacion_app`, puerto 3307	PC	Solo mediante CALL a procedimientos
Reloj	Kotlin, Compose for Wear OS 1.4.1, OkHttp, DataStore	Emulador Wear OS Small Round (API 34)	`10.0.2.2:3000`
Chatbot	Azure App Service + Foundry	Nube	`https://agerecovery-api-7m4q9x.azurewebsites.net` (POST `/api/chat`)

Un iPhone no se empareja con Wear OS. Por eso el reloj y el teléfono no se comunican entre sí: ambos pasan por el backend. La demostración consiste en marcar un cuidado en el reloj y verlo marcado en el iPhone.

3. Estado de cada componente

3.1 Base de datos (`recuperacion_app`)

- Se usan los procedimientos reales: `sp_login_paciente`, `sp_mi_plan`, `sp_cuidados_del_dia`, `sp_marcar_cuidado`, `sp_guardar_foto`, `sp_notificar_alarma`.
- El SQL original no se modificó.
- Tablas confirmadas: `alerta`, `clinica`, `cuidado_plan`, `cuidado_realizado`, `etapa_plan`, `foto_progreso`, `intento_login` y otras (14 en total).
- Paciente demo: código `123456` (plan de Bioestimulación facial, día 3 de 7).

- Limpieza ya ejecutada: `cuidado_realizado`, `alerta`, `foto_progreso` e `intento_login` quedaron en 0 filas.
- Bloqueo de login: 60 s tras 5 fallos. Los 8 intentos acumulados durante las pruebas ya se borraron.

3.2 Backend (`4-Backend`)

- Endpoints: `POST /api/login`, `GET /api/plan`, `GET /api/cuidados/hoy`, `POST /api/cuidados/:id/marcar`, `POST /api/fotos`, `GET /api/fotos`, `GET /api/fotos/:id`, `POST /api/alarmas`, `GET /api/salud`.
- `DB_PORT` configurable (3307 en esta PC).
- Fotos: solo JPEG y PNG, máximo 5 MB, nombre UUID, validación por firma del archivo, guardadas en `uploads/`, servidas solo a su dueño con token.
- Cada foto se registra siempre en MySQL con `sp_guardar_foto`. Como el SQL no tiene procedimientos para listar o consultar fotos, existe un índice JSON privado (`fotos-index.json`, en la raíz de `4-Backend`) que guarda id de API, paciente, plan, ruta y fecha. Escritura atómica (archivo temporal y renombrado) y serializada.
- Ánimo: campo opcional de 1 a 5. Si no viene, se usa 3 y queda marcado como `TODO_MEDICO` en el código.
- Alarmas: `p_id_senal = NULL` y el síntoma opcional va en `p_mensaje` (máximo 300 caracteres).
- Cambio reciente: `POST /api/cuidados/:id/marcar` acepta `origen` opcional (`telefono` o `reloj`, lista blanca exacta; por defecto `telefono`; otro valor devuelve 400 con "Origen inválido."). Probado con PowerShell.

3.3 App móvil (`3-AppMovil-Expo`)

- Login real con código de 6 dígitos; token en `expo-secure-store`.
- Inicio con tratamiento, "Día X de Y", etapa y cuidados de hoy con botón "Hecho" (actualización optimista con reversión si falla).
- Fotos: `expo-image-picker`, compresión a un ancho máximo de 1280 px con `expo-image-manipulator` (JPEG), subida con FormData, descarga con cabecera Authorization.
- Corrección aplicada: en Expo SDK 58 el fetch global no acepta `{uri, name, type}` en FormData. Ahora se usa `new ExpoFile(uri)` de `expo-file-system`.

- Alarma: cuenta regresiva de 5 s con cancelación, `POST /api/alarmas` y botón que abre el marcador (`tel:911`) sin llamar directo.
- `app.json`: `usesCleartextTraffic: true` para permitir http hacia el backend local.
- Chat conectado al chatbot de Azure: el estado de conversación solo vive en memoria; incluye reintento y tarjeta de Plan B (WhatsApp 6538492893 y llamada 6535158028); el aviso "Asistente de IA · no sustituye al médico" es fijo.
- Respuestas del chat recortadas en pantalla (alrededor de 280 caracteres) con botón "Ver más / Ver menos". El recorte es solo visual.
- Aviso "Contenido de ejemplo, pendiente de revisión médica" controlado por la bandera `SHOW_SAMPLE_CONTENT_NOTICE` en `config.ts`.
- Todo el trabajo queda en local. No se ha hecho `git push` (el chatbot lo están revisando personas de Azure).

3.4 Reloj (`sos-wear-os-main`, módulo `app/`)

- Se reutilizó el proyecto SOS Wear anterior. Se creó el flujo nuevo sin borrar el código viejo (que quedó sin uso): `BackendConfig.kt`, `CareApi.kt`, `CareViewModel.kt`, `CareScreens.kt`, `AgeRecoveryApp.kt`. Las carpetas `backend/` y `test/` del proyecto viejo no se tocaron.
- Pantallas: login con teclado numérico, inicio con tarjetas verticales por cuidado (botón "Marcar hecho" o indicador "Hecho ✓"), y alarma con cuenta regresiva de 5 s, "Cancelar", envío y aviso de llamar al 911.
- Marcar un cuidado envía `{"origen":"reloj"}`. Si falla, se revierte y se avisa.
- Red: `network_security_config` que permite http solo para `10.0.2.2` (se quitó la habilitación global de tráfico sin cifrar).
- Compilación: `gradlew assembleDebug` correcta con JDK 21 (`C:\Users\Vlad-pc\.jdks\temurin-21.0.12.1`). En Android Studio, el Gradle JVM debe ser 21 (el 25 no es compatible con Gradle 8.13).
- Verificado en el emulador: login, plan y cuidados reales del backend; alarma funcional ("Alerta enviada").
- Último pulido pedido a Copilot (centrado de los botones, margen del texto del 911, menos interlineado): compilado, pendiente de verificar con una captura nueva.

3.5 Chatbot de Azure

- Sigue activo y responde bien desde la app. La cuenta del curso podría desactivarse pronto; en el historial de Azure se advertía de esa desactivación.

- Respuestas demasiado largas: la causa es que el agente no seguía el límite de estilo. En la carpeta local `agerecovery-chatbot-foundry-main` ya está corregida la sección ESTILO de `agent/instrucciones.md` (máximo 3 párrafos, alrededor de 80 palabras, pregunta aclaratoria si no se dice el tratamiento).
- **Ese cambio no está publicado.** Publicarlo requiere acceso a Azure (`az login` y `npm run add-knowledge` con las variables de Foundry). Sin acceso a Azure no se puede aplicar. Por eso se agregó el recorte visual "Ver más" en la app como parche.
- Se borraron de EXAMEN las carpetas `1-SitioWeb`, `2-Chatbot`, `Documentacion` y `EXAMEN.rar` por decisión del alumno. El intento de respaldo en ZIP desde CMD falló por la protección de Windows (Acceso controlado a carpetas). No está confirmado que exista un respaldo. Conviene verificarlo.

4. Problemas ya resueltos (cronología resumida)

1. **Chat decía "Sin conexión"**: se comprobó con PowerShell que el chatbot de Azure funcionaba bien (conversación de dos turnos). El fallo estaba en la app. Se corrigieron el registro de errores en desarrollo y el manejo de timeouts. Después el chat respondió correctamente.
2. **Subida de fotos con error de FormData**: resuelto con `ExpoFile` (ver 3.3).
3. **Botón de alarma encimado con la barra de estado del iPhone**: se pasó el prompt a Copilot (área segura superior). Pendiente confirmar el resultado en pantalla.
4. **Bloqueo de login por intentos fallidos**: limpiado en `intento_login`.
5. **Reloj con textos montados**: rediseño con tarjetas verticales y chips. Compila y se ve ordenado.
6. **Backend sin origen `reloj`**: añadido y probado.

5. Problema en curso: la lista de fotos

Síntoma. Tras subir una foto, la app muestra "Foto enviada correctamente" y debajo, en rojo, "La foto se subió, pero no se pudo actualizar la lista".

Causa encontrada. Tres lugares guardan fotos y estaban desincronizados:

Lugar	Fotos
MySQL `foto_progreso`	2 (subidas a las 03:34:27 y 03:34:43, probable resubida tras ver el error)

Lugar	Fotos
Carpeta `uploads/`	2 (las viejas se habían borrado a mano)
Índice `fotos-index.json`	4 (2 viejas de pruebas con curl más las 2 nuevas)

`GET /api/fotos` devuelve las 4 del índice. Al descargar las 2 viejas, el archivo ya no existe y el backend responde `404 Foto no encontrada`. La consola de Expo repite `Photo download HTTP 404`. Una descarga fallida basta para que la app diga que no pudo actualizar la lista.

Ya hecho. `foto_progreso` quedó en 0 filas.

Pendiente (con el backend apagado):

1. En el Explorador de archivos, borrar `4-Backend\fotos-index.json` (si no existe, el servidor arranca con índice vacío).
2. Vaciar los archivos de `4-Backend\uploads` sin borrar la carpeta.
3. Arrancar el backend con `npm start`.
4. Cerrar y abrir la app del iPhone; en Seguimiento la lista debe verse vacía y sin error.
5. Subir **una sola** foto, esperar y no tocar nada. Debe aparecer en la lista. La consola de Expo no debe mostrar 404.

Usar el Explorador y no CMD: Windows Security (Acceso controlado a carpetas) bloqueó antes a `cmd.exe` en el Escritorio.

Mejora recomendada. Pedirle a Copilot (solo en `4-Backend`) que `GET /api/fotos` ignore las entradas del índice cuyo archivo ya no exista, para que un archivo faltante nunca rompa la lista.

6. Cómo encender todo

1. **MySQL** encendido (Workbench conectado a la instancia local).
2. **Backend**, en `4-Backend`: `npm start`. Comprobar con `Invoke-RestMethod http://192.168.1.14:3000/api/salud` (debe responder `ok: True`).
3. **Expo**, en `3-AppMovil-Expo`: `npx expo start --clear` y escanear el QR con el iPhone en la misma Wi-Fi.
4. **Reloj**: abrir `sos-wear-os-main` en Android Studio, encender el emulador Wear OS Small Round y presionar Run.

Si la IP de la PC cambia (`ipconfig`), actualizar `API_BASE_URL` en `src/config.ts` . Hoy es `192.168.1.14` .

7. Guion de la demo (un registro de cada cosa)

1. iPhone: iniciar sesión con `123456` y mostrar plan y cuidados.
2. Reloj: tocar "Marcar hecho" en un cuidado.
3. iPhone: recargar Inicio y mostrar que ese cuidado aparece marcado.
4. iPhone: subir una foto en Seguimiento y verla en la lista.
5. Chat: preguntar `¿Cuánto dura el resultado?` y usar "Ver más".
6. Reloj: enviar una alarma dejando que la cuenta llegue a 0.

Antes de la presentación real, repetir la limpieza: `DELETE` en `cuidado_realizado` , `alerta` , `foto_progreso` e `intento_login` (con `SET SQL_SAFE_UPDATES = 0` antes y `= 1` después), vaciar `uploads/` y borrar `fotos-index.json` con el backend apagado.

8. Pendientes y puntos abiertos

- Terminar el arreglo de la lista de fotos (sección 5).
- Verificar con captura el pulido del reloj y el botón de alarma en el iPhone (el ícono azul de engranaje que aparece encima es el menú de desarrollo de Expo Go, no la app).
- Probar las preguntas de seguridad del chat: dosis, precio, hinchazón con fiebre, tema ajeno a la clínica.
- Publicar las instrucciones cortas del agente si se recupera el acceso a Azure.
- Confirmar si existe respaldo de `1-SitioWeb` , `2-Chatbot` y `Documentacion` , y si la materia exige esos entregables.
- Marcadores `TODO_MEDICO` : ánimo neutral 3 por defecto, señales de alarma del FAQ aún locales en la app, y contenido de ejemplo pendiente de revisión médica.
- Para quitar el índice JSON de fotos a futuro, recomendar agregar al SQL procedimientos para listar fotos del paciente y consultar una por id.
- Nada se ha subido a Git: se mantiene todo local por ahora. Antes de cualquier `git add` , revisar con `git status` que no entre ningún `.env` .

9. Reglas de trabajo con Copilot

- Cada prompt delimita la carpeta donde puede trabajar y qué no debe tocar (por ejemplo, solo `4-Backend`, solo `app/` del reloj).
- Se pide primero leer y avisar qué se va a cambiar en los cambios grandes.
- Siempre se cierra con un chequeo: `npx tsc --noEmit` en la app, `node --check server.js` en el backend, `gradlew assembleDebug` en el reloj.
- Si Copilot pide abrir una URL, aceptar solo dominios de documentación oficial (`docs.expo.dev`, `reactnative.dev`). En una URL ajena o innecesaria, elegir Skip.